Software Test Plan (STP) - Dynamic FAQ Builder

Project: Dynamic FAQ Builder
Version: 1.0
Authors: <QA Team>
Date: 31-08-2025
Status: Sample / Draft

1. Introduction

Purpose:
This document defines the Test Plan for the FAQ Builder System v1.0. It ensures that all functional and non-functional requirements from the SRS are validated through systematic testing.

Scope:
Testing will cover the Admin dashboard (CRUD operations, tagging, workflow), the End-user search UI (search, filters, feedback), and supporting modules (authentication, logging, analytics, backups). Performance, security, and compliance requirements will also be validated.

Excluded from scope: AI-based semantic search and third-party knowledge base integrations.

References

- Software Requirements Specification (SRS) – FAQ Builder v1.0
- Requirements Traceability Matrix (RTM) – FAQ Builder v1.0
- Project Design Documentation (to be delivered by development team)

Definitions

- **CRUD** – Create, Read, Update, Delete
- **RBAC** – Role-Based Access Control
- **JWT** – JSON Web Token
- **UI** – User Interface
- **CTR** – Click-Through Rate
- **RTM** – Requirements Traceability Matrix

- **API** – Application Programming Interface

2. Test Items

The following system modules/features will be tested:

- Q/A Content Management (create, update with versioning, archive/delete, bulk import, tag management, publish workflow)
- Search & Ranking (keyword ranking, filters, highlights, typo tolerance, reindexing)
- Suggestions & Feedback (autocomplete, helpful/not helpful tracking)
- End-User UI (filters, canonical answer pages, pagination)
- Audit & Compliance (logs, export of logs)
- Analytics & Reporting (dashboard metrics, report builder)
- Reliability & Ops (health checks, backups, restore)
- Security (input sanitization, brute force prevention, access restrictions)

3. Features to be Tested

- Features mapped to SRS requirement IDs:
  QA-F-001 to QA-F-003: Authentication & RBAC (login, roles, SSO)
- QA-F-010 to QA-F-015: Q/A CRUD operations & workflow
- QA-F-020 to QA-F-025: Search & Ranking
- QA-F-030 to QA-F-032: Suggestions & Feedback
- QA-F-040 to QA-F-041: End-User Search UI
- QA-F-050 to QA-F-051: Audit & Compliance
- QA-F-060 to QA-F-061: Analytics & Reporting
- QA-F-070 to QA-F-071: Reliability & Operations
- QA-F-080 to QA-F-082: Security
- FAQ-NF-001 to FAQ-NF-005: Non-functional (performance, uptime, security, auditability, usability)

4. Features Not to be Tested

- AI-based semantic search (explicitly excluded in SRS scope)
- Third-party knowledge base integrations
- External notification services (optional, not core)
- Hosting infrastructure and vendor-managed cloud services
- Accessibility beyond WCAG 2.1 AA baseline (e.g., advanced localization, optional internationalization features)

5. Test Approach / Strategy

Levels:

- Unit tests (module-level)*: Individual components (Auth Module, Q/A CRUD API, Search API, Indexing Engine, Analytics Dashboard)

- Integration tests (API ↔ Search Engine)*: FAQ system integration with indexing engine (Elasticsearch/Lucene), database connectivity, authentication service integration

- System tests (end-to-end FAQ functionality)*: Complete user workflows from Q/A creation to search and feedback

- Acceptance tests (UAT)*: Admin dashboard usability, end-user search experience validation

Types:

- Functional testing (core features)*: Q/A CRUD operations, search functionality, role-based access control, feedback system

- Regression testing*: Automated test suite execution after code changes, especially for search indexing and ranking

- Performance testing (response time, load)*: Search query response times (<2 seconds), concurrent user load testing, indexing performance

- Usability testing (UI clarity, accessibility)*: Admin dashboard workflow efficiency, end-user search interface, WCAG 2.1 AA compliance

- Security testing*: Authentication flows, input sanitization, API rate limiting, role-based permissions

Entry Criteria:

- Stable build delivered with all core modules integrated

- Test data available (sample Q/A pairs, user accounts with different roles)

- Test environment ready (database, search engine, web server configured)

- Test cases reviewed and approved

Exit Criteria:

- 100% of planned test cases executed

- 0 critical defects open, <5% medium-priority defects

- All acceptance criteria satisfied per RTM

- Performance benchmarks met (95% of queries <2 seconds)

- Security validation completed with no high-risk vulnerabilities

5.1 Security Validation

Security is a top priority for our FAQ Builder system, especially since it handles user authentication and potentially sensitive organizational knowledge. Our security testing approach covers multiple layers of protection.

Authentication & Authorization Testing

We'll thoroughly test the login process to ensure JWT tokens are properly generated and validated. Role-based access control is critical - admins should have full access, editors can create and modify content but not delete, and viewers can only read. We'll also test the SSO integration to make sure external authentication providers work seamlessly with our system.

Input Sanitization & Injection Prevention

Since admins can enter rich text content for answers, we need to ensure malicious scripts can't be injected. We'll test XSS prevention by attempting to inject various forms of malicious JavaScript code. For the search functionality, we'll test SQL injection attempts to verify our query handling is secure. The bulk import feature will be tested with malformed files to ensure our validation catches problematic content.

Network Security Validation

All communication must use TLS 1.2 or higher encryption. We'll verify certificates are properly configured and that no sensitive data is transmitted in plain text. API endpoints will be tested to ensure they reject connections that don't meet our security standards.

API Security & Rate Limiting

We'll test that our API rate limiting actually works by sending rapid-fire requests and confirming they get blocked appropriately. Brute-force login attacks will be simulated to verify our lockout mechanisms activate correctly. We'll also attempt to access restricted endpoints without proper authorization to ensure our security controls are effective.

Data Protection & Audit Compliance

The integrity of our audit logs is crucial for compliance. We'll verify that all admin actions are properly logged and that these logs cannot be tampered with. If GDPR applies, we'll test data export and deletion capabilities to ensure user privacy rights are respected.

Penetration Testing

Professional security testing will include attempting to find vulnerabilities in our search engine integration, testing for weaknesses in the admin panel, and validating that user sessions are managed securely throughout the application.

6. Test Environment

Our testing environment needs to closely mirror the production setup while providing the flexibility needed for thorough testing.

Hardware Infrastructure

We'll set up dedicated servers for our web application, database, and search engine. If load balancing is part of the architecture, we'll include that in our test environment as well. The setup should handle the expected user load during testing without performance bottlenecks that could skew our results.

Software Components

The core FAQ Builder application version 1.0 will be deployed along with our chosen database system and search engine. We'll test across multiple web browsers including Chrome, Firefox, Edge, and Safari to ensure cross-browser compatibility. Mobile browsers will also be included since the system needs to be responsive across different devices.

Testing Tools Suite

For automated UI testing, we'll use Selenium WebDriver and Cypress to simulate user interactions. API testing will be handled through Postman for manual testing and REST Assured for automated API validation. Performance testing will use JMeter and Artillery to simulate various load conditions. Security testing will leverage OWASP ZAP and Burp Suite for vulnerability scanning. All defects will be tracked in Jira, and we'll use TestRail for overall test management and reporting.

Test Data Preparation

We'll create a comprehensive dataset including 50-100 sample Q/A pairs covering various topics, tags, and languages to test search functionality thoroughly. User accounts will be set up for each role type (admin, editor, viewer) to test permission controls. We'll prepare a collection of search queries ranging from common user questions to edge cases and intentionally malformed inputs. Test files for bulk import functionality will include both valid and invalid data to verify our validation processes.

7. Test Schedule

Milestones:
- **Test case design:** September 5, 2025

Environment setup: September 7, 2025

Test execution start: September 8, 2025

Test execution end: September 20, 2025

User Acceptance Testing (UAT): September 22-25, 2025

8. Test Deliverables

Throughout the testing process, we'll produce comprehensive documentation and artifacts that serve multiple purposes: guiding our testing activities, providing evidence of thorough validation, and creating valuable resources for future maintenance and updates.

Test Plan Document

This comprehensive document (the one you're reading now) serves as our testing roadmap. It outlines our entire approach, defines what we'll test and how, establishes our quality gates, and provides the schedule everyone will follow. The test plan will be a living document, updated as we learn more about the system and refine our approach based on early testing results.

Test Cases Documentation

We'll create detailed test cases covering every functional requirement identified in the SRS document. Manual test cases will provide step-by-step instructions for human testers, including expected results and pass/fail criteria. For example, we'll have test cases like "TC-AUTH-01: Verify JWT Token Generation" and "TC-QA-01: Create Q/A Pair with Required Fields." Automated test cases will be documented with clear descriptions of what each script validates, making it easy for developers to understand what broke when a test fails.

Test Scripts and Automation Suite

Our automated test scripts will be developed using modern frameworks like Selenium for UI testing and REST Assured for API validation. Each script will be well-commented and modular, making them easy to maintain and update. We'll organize scripts by functionality area (authentication, Q/A management, search, etc.) and include both positive test scenarios

(happy path) and negative testing for error conditions. Performance test scripts will simulate realistic user loads and measure response times across different system components.

Test Data Collections

We'll maintain several categories of test data to support thorough validation. This includes a curated set of Q/A pairs representing different content types, lengths, and complexity levels. User account data will cover all role types with various permission combinations. Search query datasets will range from simple keyword searches to complex boolean queries and edge cases like special characters or extremely long search terms. We'll also prepare test files for bulk import functionality, including both valid data and intentionally malformed files to verify our error handling.

Test Execution Logs and Evidence

Every test execution will be documented with detailed logs showing what was tested, when it was performed, who conducted the test, and what the results were. Screenshots will capture both successful operations and any defects discovered during testing. Performance test results will include detailed metrics on response times, throughput, and system resource utilization. Security test logs will document all vulnerability scans and penetration test attempts, providing a complete audit trail of our security validation efforts.

Defect Reports and Tracking

All discovered issues will be documented in our defect tracking system (Jira) with consistent formatting and detailed information. Each defect report will include a clear description of the problem, steps to reproduce the issue, expected versus actual behavior, severity and priority classifications, affected system components, and supporting evidence like screenshots or log files. We'll track defects through their entire lifecycle from discovery through resolution and verification of fixes.

Test Summary and Final Report

Our comprehensive test summary report will provide a complete picture of testing activities and results. This executive-level document will summarize what was tested, highlight key findings, present metrics on test execution and defect discovery, assess whether the system meets its acceptance criteria, and provide recommendations for production readiness. The report will include visual dashboards showing test coverage, pass/fail rates, defect trends, and performance benchmarks, making it easy for stakeholders to understand the system's quality status at a glance.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Karthikeya	Define test strategy, manage plan & schedule, coordinate execution, ensure quality gates.
Test Engineer	Ishika	Design & run test cases, prepare test data, log defects, maintain evidence.
Developer	Yeshwanth	Fix defects, support integration, assist in triage and regression testing.
Product Manager	Jayanth	Validate features, review results, approve UAT, sign off release readiness.

10. Risks and Mitigation

Risk	Mitigation
Unstable build or late code delivery	Define clear entry criteria; only accept stable builds for testing; buffer time in schedule.
Incomplete or invalid test data	Prepare comprehensive Q/A datasets, role accounts, and malformed inputs in advance.
Performance issues under load	Conduct load and tune database and indexing engine.

11. Assumptions & Dependencies

- Stable build and required test data will be available before execution
- All testing tools and resources will be ready and accessible.
- Development team support for defect fixes and integration

12. Suspension & Resumption Criteria

Suspend testing if:

- Critical defects block core workflows (e.g., login, CRUD, search).
- Environment becomes unstable or unavailable.

Resume testing if:
- Blocking defects are fixed and verified
- Environment is restored and passes basic sanity checks.

13. Test Case Management & Traceability

RTM ensures mapping of SRS requirements to test cases.
Example:
- ATM-F-001 (PIN validation) → TC-Auth-01, TC-Auth-02
- ATM-F-010 (Withdrawal) → TC-WD-01, TC-WD-02
- ATM-NF-001 (Response time) → TC-Perf-01

14. Test Metrics & Reporting

Metrics collected:
- % test cases executed
- % passed/failed
- Defect density
- Defect aging
- Requirement coverage

Reports:
- Daily execution status
- Final Test Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead		
Dev Lead		
Product Owner		

16. Test Cases – Dynamic FAQ Builder

Test Case ID	Test Scenario / Description	Preconditions	Test Steps / Input Data	Expected Result	Postconditions / Remarks
TC-AUTH-01	Verify login with valid credentials (QA-F-001)	User account exists in DB with valid email and password hash	1. Open Login page → enter valid email & password → Submit	System returns HTTP 200 and issues JWT token stored in session storage	User redirected to dashboard
TC-AUTH-02	Verify invalid login attempt (QA-F-001)	User account exists	1. Enter wrong password → Submit 3 times	System returns 401 “Invalid credentials”; account locked temporarily	Login disabled for configured duration

TC-AUTH-03	Verify RBAC access control (QA-F-002)	Admin and User accounts exist with different roles	1. Login as Admin → access /admin ✅ 2. Login as User → try /admin ❌	Admin allowed; User gets 403 Forbidden	Role-based permissions enforced
TC-QA-01	Create Q/A pair (QA-F-010)	Logged in as Editor or Admin	1. Click “Add Q/A” → enter question, answer, tags → Save	Record created (201 response); visible in list and index	New entry stored in DB
TC-QA-02	Update existing Q/A (QA-F-011)	Q/A exists in DB	1. Edit answer field → Save	Version history updated; previous version retained	Change reflected in list
TC-QA-03	Archive Q/A pair (QA-F-012)	Published Q/A exists	1. Click “Archive” button	Status changes to Archived and hidden from search	Item hidden from end-user UI

TC-QA-04	Bulk import Q/A (QA-F-013)	CSV file with valid & invalid rows prepared	1. Upload CSV → Run Import	Valid rows added; invalid rows listed with line numbers and reasons	Validation report stored
TC-QA-05	Manage tags (QA-F-014)	Tags exist in system	1. Merge “Payment” and “Billing” tags	All Q/As retagged; index rebuilt successfully	Tag list updated; no duplicates
TC-QA-06	Publish workflow (QA-F-015)	Draft Q/A exists	1. Click “Submit for Review” → “Publish”	Status changes to Published and visible in search	Workflow state transition logged
TC-SRCH-01	Keyword search (QA-F-021)	Indexed Q/A data available	1. Search “password reset”	Top results show relevant answers ranked by relevance score within ≤ 2 s	Search results returned accurately

TC-SRCH-02	Search with filters (QA-F-022)	Index built and active	1. Apply filter tag:“billing”, date range	Filtered results only displayed correctly	No errors for empty filters
TC-SRCH-03	Typo tolerance search (QA-F-024)	Index ready	1. Search “accunt setup”	Results for “account setup” displayed (edit-distance ≤ 1)	System returns closest matches
TC-SUG-01	Autocomplete suggestions (QA-F-030)	Search index ready	1. Type “bill” in search bar	Suggestions appear < 150 ms; click opens answer	Suggestion cache updated
TC-SUG-02	Helpful / Not-helpful feedback (QA-F-031)	Logged in user viewing answer	1. Click “Helpful” or “Not Helpful”	Feedback recorded in DB with user ID and timestamp	Analytics updated successfully

TC-AUD-01	Verify audit log entries (QA-F-050)	Admin login active	1. Perform CRUD operations (Create/Edit/Delete)	Each action logged with user, timestamp, and action type	Log file immutable and exportable
TC-SEC-01	SQL / XSS input sanitization (QA-F-080)	App running with test user session	1. Inject `<script>` in answer field and submit	Input neutralized; no malicious code executed	Validation error message displayed
TC-PERF-01	Search response time (FAQ-NF-001)	System deployed with sample data	1. Run 1000 queries via JMeter	≥ 95% queries respond ≤ 2 seconds	Meets performance benchmark
TC-REL-01	Backup and restore (QA-F-071)	Backup scheduler configured	1. Trigger manual backup → restore to staging	Data restored without loss or corruption	Backup validated successfully
TC-SEC-02	HTTPS enforcement (FAQ-SR-001)	App hosted on test server	1. Attempt HTTP access	Redirects to HTTPS / rejects plain HTTP	TLS configuration verified

TC-UX-01	Page load time on 4G (FAQ-NF-005)	Front-end deployed on 4G test network	1. Load home page on Chrome DevTools (4G throttle)	Page loads ≤ 3 seconds	Meets usability and UI performance criteria